Ders 6 Genişletilmiş Notu: Goldbach Kodları, Prosesin Bellek Bölgeleri, Algoritma Karşılaştırma ve Veri Yapılarına Giriş

Ders 6 Genişletilmiş Notu: Goldbach Kodları, Prosesin Bellek Bölgeleri, Algoritma Karşılaştırma ve Veri Yapılarına Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Goldbach Önermesinin Derste Kullanılan Biçimi
- 2. Birinci Kod: Asal Deposu, Karekök Yöntemi ve İki Uçtan Arama
- 3. Asalları Saklamak mı, Yeniden Hesaplamak mı?
- 4. İkinci Kod: Aramayı Daraltmak
- 5. Üçüncü Kod: Tek Sayılar, Gereksiz Bayrak ve İkili Arama
- 6. Büyük Dizilerde Bellek Hatası ve Prosesin Bellek Bölgeleri
- 7. Dördüncü Kod: Fonksiyon Kullanmak
- 8. Algoritmaları Karşılaştırmak
- 9. Mesleki Derslere Yaklaşım ve Derleyici Farkları
- 10. En Büyük Not Problemi
- 11. En Büyük İkinci Not
- 12. Algoritmanın Tıkandığı Yer
- 13. Veri Soyutlama ve Dizi
- 14. Otomata Modelleri ve RAM Modeli
- 15. Dizi Bize Ne Kazandırır?
- 16. Matris, Graf ve Ağaç
- 17. Otomata Türleri ve Veri Yapılarının Önemi
- 18. Adresler ve Çok Boyutlu RAM Soruları

1. Goldbach Önermesinin Derste Kullanılan Biçimi

Dersin ilk yarısında öğrencilerin Goldbach problemi için yazdığı kodlar ekran paylaşımıyla incelendi. Goldbach önermesi, çift sayıların iki asal sayının toplamı olarak yazılabileceğini söyler. Önce bir öğrencinin sorusu üzerine problemin ne olduğu netleştirildi, çünkü asıl önerme ile derste istenen şey aynı değildi.

Soru: Program, çift sayıların iki asal sayının toplamı olduğunu buluyor mu? **Cevap:** Asıl önerme zaten çözülmüş değil. Biz çözebilseydik adımız bir yerlere yazılırdı. Önerme bütün çift sayılar için konuşuyor. Biz önermeyi kendimiz değiştirdik: Girilen bir N sayısına kadar, örneğin 300'e kadar olan çift sayılarda bu önerme doğru mudur? Yazılması istenen kod, bu değiştirilmiş önermeyi denetleyen koddur.

Hoca, önermenin doğruluğu da yanlışlığı da kanıtlanmadığını, yanlışlığı gösterilemediği için çok büyük sayılarda doğru kabul edildiğini söyledi. Öğrencilerin çalıştırdığı denemelerde de karşıt örnek çıkmadı.

Problemin yapısı bir tasarım kararını da belirliyor. Bir sayı birden fazla şekilde iki asalın toplamı olabilir. Soru yalnızca önermenin sağlanıp sağlanmadığı olduğu için bir sayı için tek bir uygun çift bulmak yeterlidir. Hocanın sözüyle, ilk kanıtladığımız yerde çıkıp devam etmememiz gerekir. Bu da döngüden erken çıkmayı sağlayan bir yapı gerektirir.

2. Birinci Kod: Asal Deposu, Karekök Yöntemi ve İki Uçtan Arama

2.1 Asal Deposu

İlk öğrenci, programını çalıştırıp 1000 girerek her çift sayının hangi iki asalın toplamı olduğunu ekranda gösterdi. Bir sayı için tek bir çift üretiyordu. Hoca bunun sorunun beklentisine uygun olduğunu söyledi.

Hoca burada başka bir öğrenciye "Dizi kullanmadan bu soru çözülebilir miydi?" diye sordu. Öğrencinin cevabı, bütün asalları değerlendirmek gerektiği için dizisiz çözümün olsa bile çok uzun ve zor olacağı yönündeydi. Hoca, iki öğrencinin de aynı şeyi yaptığını, yani bir **asal sayı deposu** oluşturduğunu belirtti.

Bu depo bir performans kazancı sağlar. Depo olmazsa her çift sayı için o sayıya kadarki asalları yeniden bulmak ve toplamları yeniden denemek gerekir. Oysa önce 4'e bakarken 4'e kadarki asallar bir kenara yazılır. Sonra 6'ya bakılırken 4'e kadarki asallara zaten bakılmıştır, yalnızca 6'ya kadar yeni çıkanlar diziye eklenir. Hoca bunu bir torbaya veya kümeye benzetti: Buldukça torbaya ekliyoruz. Koddaki `primeCount` benzeri sayaç değişkeni de 4 için test yapılırken kullanılıyor, 6 için de kullanılmaya devam ediyor ve sürekli artıyordu.

2.2 Programın Başlangıcı ve Karekök Yöntemi

Kodun başlangıcı şöyle kuruluyordu:

1. Girilen sayı okunur.
2. Asal dizisinin 0. elemanına 2 yazılır.
3. Kontrol edilecek ilk çift sayı olarak `n` 4 yapılır. Asal adayları için kullanılan değişken 3'ten başlar.
4. Bayrak (flag) 1 yapılır. Bayrak 1 olduğu ve `n` girilen sayıdan küçük olduğu sürece dış `while` döner.

Asal dizisi güncellenirken öğrencinin internette bulduğunu söylediği karekök yöntemi kullanılıyordu. Bir sayının asal olup olmadığını öğrenmek için karekökü alınır, üste yuvarlanır ve o değere kadarki asal sayılara bölünüp bölünmediğine bakılır. Bölünüyorsa sayı asal değildir, bölünmüyorsa asaldır. Hoca, bunun için kullanılan `while` döngüsünü ve `isPrime` adlı bayrağı "güzel bir flag" diye nitelendirdi: Karekökten küçük sayılardan birine bölünüyorsa bayrak döngüyü durdurur ve sayı asal kabul edilmez. Asalsa diziye eklenir.

2.3 İki Uçtan Arama

Asallar güncellendikten sonra bayrak 0 yapılır ve `k` değişkenine 0 atanır. Öğrenci asal dizisinde bir baştan, bir sondan karşılaştırma yapıyordu. Dıştaki `k` dizinin başından ilerler. İçteki `j` ise dizinin sonundan, yani o ana kadar bulunan son asaldan geriye doğru iner. Öğrenci sondan başlamasının sebebini gözlemiyle açıkladı: Bir küçük asal ile çok büyük bir asalın toplamı genellikle hemen sonucu veriyordu.

Bayrak 0 iken ve toplam, sayıdan büyük veya ona eşitken eşitlik kontrol edilir. Eşitse bayrak 1 yapılıp döngüden çıkılır, değilse `j` azaltılır.

Soru: Bu kısma `else` yazılabilir miydi? **Cevap:** Yazmaya gerek yok. Eşitlik sağlandığında zaten döngüden çıkılıyor, sağlanmadığında `j` azaltılıyor.

Bulunan çift ekrana yazılır ve `n` 2 artırılarak 4'ten 6'ya geçilir. Hoca araya girerek, modüler programlamaya ve fonksiyonlara geçince bu kodların daha oturaklı ve düzgün yazılacağını, şu an aynı şeyler tekrar ediliyormuş gibi göründüğünü ekledi.

2.4 Son Kontrol

Bütün dış döngü bittikten sonra bayrak hâlâ 1 ise o sayıya kadar bütün çift sayıların Goldbach önermesini sağladığı yazılır. Bayrak 0 çıkarsa önermenin hangi sayıda sağlanmadığı bildirilir.

Hoca sınıfa, 61. satırdaki `flag == 1` yerine başka ne yazılabileceğini sordu. Cevap, döngü değişkeni `n` ile girilen sayı arasında bir ilişki kurmaktı. `n` girilen sayıya ulaştıysa, demek ki sonuna kadar aranmış ve önermeyi çürüten bir durumla karşılaşılmamıştır. Denemede hata çıktı, çünkü `n` döngü içinde bir kez daha 2 artırılıyordu. Bu yüzden karşılaştırmada `n - 2` kullanmak gerekti. Düzeltmeden sonra kod doğru çalıştı.

3. Asalları Saklamak mı, Yeniden Hesaplamak mı?

Kodlar arasında bir öğrenci, algoritmanın sınırına dair bir soru sordu.

Soru: Önerme sonsuza gidiyor, henüz bir karşıt örnek bulunamamış. Sonsuza giderken diziye sürekli asal eklemek bellekte sorun çıkarmaz mı? **Cevap:** Bizim algoritmamız sonsuza gitmez, çünkü bir sayı veriyoruz. Ama çok büyük sayılarda bu gerçekten bir problemdir. Bellekte o sayıya kadarki asalları saklamak gerekir. Bellek yoksa zamandan kayıp verilir. Örneğin 100 trilyon gibi bir sayı girdiğimizi düşünelim. Önceki asallar saklanamıyorsa her seferinde yeniden hesaplanmak zorundadır.

> **Ders sonrası not:** Hoca burada “asal çarpanları” ifadesini kullandı. Bu problemde gereken, sayının çarpanları değil, büyük olasılıkla o sayıya kadarki asal sayılardır.

Hoca, sonsuza gitmek diye bir şeyin zaten olmadığını, problemin kanıtlanamamasının sebeplerinden birinin sonsuzlukla ilgili olması olduğunu söyledi. Bellek ise sınırlıdır. Bu yüzden şu seçim zorunludur:

- Dizi kullanırız, bellekten feragat ederiz.
- Dizi kullanmayız, zamandan feragat ederiz.

Hoca buna **ödünleşim (trade-off)** dedi. Sohbet kanalına dizisiz yazdığını ve kodun çok yavaş çalıştığını yazan bir öğrenciyi örnek gösterdi: Yavaş çalışması doğaldır, çünkü her şeyi sıfırdan yapmak gerekir.

Sınırı büyütmenin yolları da konuşuldu. 8 GB bellek yetmezse paralel hesaplama (parallel computing) ile birden fazla bilgisayar kullanılabilir. Dünyadaki bütün bilgisayarların belleğini kullanan bir sistem düşünülürse buna ızgara hesaplama (grid computing) denir. Hoca bunun teorik olarak kullanılabileceğini, ama uygulamada pek işe yaramayabileceğini söyledi.

4. İkinci Kod: Aramayı Daraltmak

İkinci öğrenci de asalların içinde arama yapıyordu ve karekök sınırına gitmese de bütün sayıları karşılaştırmaya göre oldukça verimli olduğunu söyledi. Asıl farkı, arama başlangıcını sayının büyüklüğüne göre seçmesiydi. Alınan sayı küçük bir sınırın (öğrencinin anlatımında $K/16$) altındaysa asalları birinciden başlatıyordu. Değilse $K/16$ veya $K/8$ gibi belirli bir parçadan başlatıyordu. Mantığı şuydu: 900.000’e bakarken ilk asaldan başlamaya gerek yoktur, çünkü ilk sayı küçükse ikinci sayının büyük olması gerekir. Öğrenci bu yöntemin küçük sayılarda bozulduğunu da belirtti.

Soru: Bu 16’lar bir teorem mi, yoksa bir kolaylık mı? **Cevap (öğrenci):** İlerleme bir yerden sonra yavaşlıyordu, bu yüzden denedim. Binde test ettim, hepsinde oluyordu. Benim kullandığım bir kolaylaştırma, daha büyük sayılarda kesin olur mu bilmiyorum.

Hoca bunu, istatistiksel olarak asalların dağılımına bakan yaklaşımlara benzetti. Önce bir dizi oluşturulur, sayı o dizi üzerinden denenir, sonra kalan değerlere bakılır. Örneğin 2’den 50’ye kadarki asallar, yani 47’ye kadarki asallar bir kenarda tutulur ve önce onlarla kıyaslanır. Yani bu bir hızlandırma aracıdır.

Kodun devamında 900.000’den önce 3, sonra 5, sonra 7 çıkarılıyor ve kalan asal mı diye bakılıyordu. Döngülerden çıkmak için iki kontrol değişkeni kullanılıyordu. Biri kalan değeri asal dizisinde arayan iç `while` için, biri dış `while` için. Kalan değer bir asala eşit olana kadar iç döngü dönüyor, eşitlik bulunursa toplam yazdırılıyordu. Hiç eşitlik yoksa dış kontrol değişkeni 1 yapılarak programdan çıkılıyordu.

Hoca kodu, problemin anlaşıldığını ve uygun yapıların seçildiğini göstermesi açısından beğendi. Burada `for` döngüsü kullanılmasının beklenmediğini, kontrollerin `while` ile yapılması gerektiğini vurguladı.

5. Üçüncü Kod: Tek Sayılar, Gereksiz Bayrak ve İkili Arama

5.1 Yalnızca Tek Sayılara Bakmak

Üçüncü öğrenci önce kullanıcının girdiği sayıya kadarki bütün asalları buldu. Asal dizisinin ilk iki elemanını elle verdi ve adayları 5'ten başlattı: 5, 7, 9, … Çünkü 2'den sonra çift asal yoktur. Bölenleri de teklerle sınırladı. Elimizdeki sayı çift olmadığı için çift sayıya bölmeye gerek yoktur. Bu yüzden `j` 3'ten başlıyor ve 2'şer artıyordu.

Karekök sınırını şöyle anlattı: 16'nın karekökü 4'tür. 17 ve 20 için de karekök 4 civarındadır. Sayı 4'e kadar bölünmediyse 4'ten büyük sayılara da bölünmüyordur ve asaldır. Ara değerler şöyledir (ara adım eklendi): $\sqrt{16} = 4$, $\sqrt{17} \approx 4{,}12$, $\sqrt{20} \approx 4{,}47$. 20 ise bu sınıra gelmeden bölünür, 17 bölünmez.

5.2 Döngü Şartıyla Aynı Bilgiyi Taşıyan Bayrak

Hoca burada 16. satırı durdurup bir öğrenciye sordu. Kod özetle şöyleydi:

```
while (j <= sqrt(i) && counter == 1) {
    if (i % j == 0)
        counter = 0;
    j += 2;
}
```

Hocanın akıl yürütmesi şuydu: `i % j == 0` doğru olunca `counter` sıfırlanıyor ve döngüden çıkılıyor. O hâlde `counter`'ın 1 olması, `i % j`'nin sıfırdan farklı olması demektir. Öyleyse içeride `if` yazmak yerine aynı şart doğrudan döngü koşuluna yazılabilir:

```
while (j <= sqrt(i) && i % j != 0) {
    j += 2;
}
```

Sonradan bir öğrenci bunun ne yaptığını tekrar sordu.

Soru: Mod sıfıra eşit değilken doğrudan yazdırsın mı diyorsunuz? **Cevap:** Yazdırsın demedim. `counter`'ı 0 yapan şart `i % j == 0`'dır. `while` şartı `counter == 1`'dir, yani bunun tam tersidir. O zaman şartı dolaylı yoldan göstermiş oluruz.

Hoca kazancı işlem sayarak gösterdi:

- **Bayraklı hâl:** `counter == 1` için bir karşılaştırma, `i % j == 0` için bir karşılaştırma daha, yani 2 karşılaştırma. Ayrıca `counter` için bir bellek kullanımı.
- **Bayraksız hâl:** `i % j != 0` için bir karşılaştırma. Fazladan değişken yok.

Yani bir karşılaştırma ve bir bayrak değişkeni kazanılır. Ancak hoca, 22. satırda da `counter == 1` kontrolü olduğunu görünce, oradaki kullanım için ek bir şey düşünmek gerekebileceğini söyledi ve değişikliği kodda tamamlatmadı. Şartın doğruluğunda ısrar etti: Döngünün içinde aynı kontrol yapıldığına göre `if`'e ve bayrak değişkenine ilke olarak gerek yoktur.

> **Ek bilgi (derste anlatılmadı):** Döngüden sonraki `counter == 1` kontrolü, büyük olasılıkla döngünün neden bittiğine bakılarak değiştirilebilir. Döngü karekök sınırı aşıldığı için bittiyse bölen bulunmamıştır, yani `if (j > sqrt(i))` doğruysa sayı asaldır. Bu biçim ders sonrasında 3'ten 200.000'e kadarki tek sayılarda bayraklı hâl ile karşılaştırıldı ve aynı sonucu verdi.

5.3 Neden 6'dan Başlanıyor?

Asalları bulduktan sonra öğrenci kontrolü 6'dan başlattı, 2 çıkarmayı da hiç denemedi. Sebebi şuydu: 2 yalnızca 4 için kullanılır. 6'dan itibaren çift bir sayıdan 2 çıkarılınca sonuç yine çift olur ve 2'den büyük olduğu için asal olamaz. Örneğin 8'den 2 çıkarınca 6 kalır. Bu yüzden

aramaya doğrudan 3'ten başlanabilir. Asal dizisinde `j = 1` ilk elemanı değil, 3'ü gösterir. Bir başka öğrenci de aynı yaklaşımı kullanmıştı.

5.4 İkili Arama

Öğrenci, sayıdan seçtiği asalı çıkarıp bir kalan elde ediyor, bu kalanın asal dizisinde olup olmadığını **ikili arama (binary search)** ile kontrol ediyordu. `first` 1, `last` dizinin son elemanının indisi, `mid` ise ikisinin toplamının ikiye bölümünün tam kısmıydı. Eleman bulunursa bayrak sıfırlanıp ikinci sayıya geçiliyordu. Döngü bittiğinde bayrak hâlâ aranan durumu göstermiyorsa bir karşıt örnek bulunmuş demekti ve programdan çıkılıyordu. Sonunda kaç asal kullanıldığı veya karşıt örnek yazdırılıyordu.

Öğrenci kodu yaklaşık 7 milyona kadar çalıştırdığını, 1 milyona kadar programın kısa sürede bittiğini söyledi. Asal dizisinin boyutunu ise 518.000'in üzerine çıkaramıyordu. 8 milyon girince asal sayısı bu sınırı aşıyordu. 1 milyon boyut yazınca da program hata veriyordu. Bu soru 6. bölümde cevaplandı.

> **Ders sonrası not:** Öğrenci 1 milyona kadar "yedi, sekiz bin" asal olduğunu söyledi. Ders sonrasında Python ile sayıldığında 1 milyona kadar 78.498 asal çıktı; kastedilen büyük olasılıkla "yetmiş sekiz bin"dir. Öğrencinin 8 milyon gözlemi tutarlıdır: 7 milyona kadar 476.648, 8 milyona kadar 539.777 asal vardır ve ikincisi 518.000'i geçer.

5.5 Asalları En Başta Bulmak

Hoca, öğrencinin asalları en başta toptan bulmasını tartıştı ve bunun tam yanlış da tam doğru da olmadığını söyledi. Örnek olarak 1000 girildiğini düşünelim. Program önce 4'e, sonra 6'ya bakar. 6'ya bakarken 6'ya kadarki asalların olduğu bir diziden değil, 1000'e kadarki asalların olduğu diziden arama yapar. Oysa belki 500'de önerme sağlanmayacaktı. Böyle bir olasılık vardır. O zaman 500'den 1000'e kadarki asallar gereksiz yere bulunmuş olur.

Öte yandan önerme bugüne kadar çürütülmediği için pratikte 1000'e kadarki asalları kullanmak zorunda kalırız. Bu açıdan öğrencinin çözümü doğrudur. Hoca algoritmik açıdan adım adım bulmayı daha güzel bulduğunu, ama burada sonucu etkileyen bir yanlışlık olmadığını, adım adım bulsak da aynı bellek ve zamanın harcanacağını söyledi.

6. Büyük Dizilerde Bellek Hatası ve Prosesin Bellek Bölgeleri

Öğrenciler dizi boyutunu çok büyük verdiklerinde yer ayrılamadığını, hatta hata alındığını gördü. Bir öğrenci dizinin başına `static` yazınca daha büyük boyutların çalıştığını internette bulmuştu. Sohbet kanalında bir başkası da `malloc` önerdi. Hoca bunları toplu olarak anlattı. Söylediği boyutlarla gösterimi şöyledir (`malloc` satırının tam yazımı altyazıda yok, ara adım eklendi):

```
int a[500];
static int b[5000000];
int *c = malloc(10000000 * sizeof(int));
```

6.1 Bellek Tahsisi

Program verilerini saklarken RAM kullanırız. RAM'i kullanırken aslında işletim sisteminden yer isteriz, işletim sistemi de bir yer tahsis eder. Buna **bellek tahsisi (memory allocation)** denir. İki türü vardır:

- **Statik bellek tahsisi:** Boyutun program içinde 3, 5, 10, 500 gibi sabit yazılması. `int a[500];` böyledir.
- **Dinamik bellek tahsisi:** `malloc` gibi bir yolla, pointer kullanılarak yer alınması. Pointer olmadan yapılamaz.

6.2 Proses ve Bölgeleri

Kod yazıp derlediğimiz program, çift tıklayarak veya komut satırından çalıştırıldığında bir **proses** hâline gelir. Prosesi işletim sistemi yönetir, bunun için proses kontrol bloğu (process control block) vardır. İşletim sistemi belleğin tamamını prosese vermez. Proses belleğe yerleşince kendine ayrılan alanda şu bölgeler bulunur:

- Program kodu
- İlk değeri verilmiş (initialized) değişkenler
- İlk değeri verilmemiş (uninitialized) değişkenler ve diziler
- Yığın (stack)
- Dinamik veriler için **heap** bölgesi

Hoca stack'in sınırlı bir alan olduğunu, aslında küçük olmasa da heap ile karşılaştırıldığında oldukça küçük kaldığını söyledi. Bir milyon elemanlı dizi istendiğinde orada bitişik olarak bir milyon elemanlık yer yoksa işletim sistemi yer vermez. Belki toplamda o kadar yer vardır ama bitişik değildir. Bitiştirme için bazı kontroller olsa da sistem çoğu zaman yer vermez ve yetersiz bellek benzeri bir hata alınır. Bazı programlama dillerinde bir istisna (exception) oluşur ve program kapanır. Hoca, 1 GB'lık bir yer istendiğinde bu alanın da bitişik olması gerektiğini vurguladı. Bu yüzden her zaman statik bellek kullanılamaz, dinamik belleğe ihtiyaç vardır.

> **Ders sonrası not:** Hoca stack'te "dinamik dizilerin" saklandığını ve hatanın bitişik alan bulunamamasından kaynaklandığını söyledi. Doğrusu büyük olasılıkla şudur: Fonksiyon içindeki `int a[N];` gibi yerel diziler stack'tedir, `malloc` ile alınan alan heap'tedir. Hata da bitişik alan eksikliğinden değil, stack'in boyut sınırının aşılmasından (stack overflow) gelir. Ders sonrasında Linux'ta stack 2 MB ile sınırlanarak denendi: Öğrencinin 518.000 elemanlı `int` dizisi (2.072.000 bayt) yerel olarak çalıştı, 1.000.000 elemanlı dizi (4.000.000 bayt) segmentation fault verdi, aynı dizi `static` veya `malloc` ile çalıştı. Öğrencinin gördüğü sınır, kullandığı sistemde stack'in büyük olasılıkla yaklaşık 2 MB olmasıyla uyumludur.

6.3 `static` ve `malloc` Neden Çözüyor?

`static` anahtar kelimesi değişkeni bu alan içinde stack dışında farklı bir yerde saklar ve o yerin kapasitesi biraz daha fazladır. Dolayısıyla bir çözümdür. Ama hoca `static` kullanmayı bıçağa benzetti: Dikkatli kullanmak ve özelliklerini iyi bilmek gerekir. "Static yazdım, oldu" demekle olmaz, başka sıkıntıları da vardır.

> **Ek bilgi (derste anlatılmadı):** Hocanın sözünü ettiği sıkıntılardan biri büyük olasılıkla şudur: Fonksiyon içindeki `static` değişken, fonksiyon bittikten sonra da değerini korur. Her çağrıda yeniden oluşturulmadığı için önceki çağrının değeri sonraki çağrıya taşınır. Bu özellik bilinmeden kullanılırsa beklenmeyen sonuçlar doğurabilir.

Diğer çözüm dinamik bellek tahsisi ile heap bölgesinden yer almaktır. Hoca bunun henüz görülmediğini, belki dönem sonuna doğru ve gelecek dönem Yapısal Programlama dersinde daha çok üzerinde durulacağını söyledi. Şimdilik önerisi küçük verilerle çalışmak ve `static` kullanmamaktır.

Bitişik alan gerektirmeyen bir veri yapısı da vardır: **bağlı liste (linked list)**. Dizilerin aksine veriler dağınık biçimde saklanabilir. Bu konuya daha sonra gelinecektir.

7. Dördüncü Kod: Fonksiyon Kullanmak

Dördüncü öğrenci dizi kullanmadan çözmüştü ve "daha hızlı olması için" fonksiyon da kullandığını söyledi.

> **Hocanın vurgusu:** Fonksiyon programı hızlandırmaz, modüler hâle getirir.

Öğrenci kastının "içerisi karışık olmasın" olduğunu söyleyince hoca bunun modülerlik anlamına geldiğini, bu rahatlığın "çift kişilik yatakta tek kişi yatmak gibi bir rahatlık" olmadığını söyleyerek şaka yaptı.

Kod da 6'dan başlıyordu, çünkü 2 ve 4'ü gereksiz yere denemek istemiyordu. Dış `while` sayının yarısına kadar gidiyordu. Her aday için sayı asal mı diye bakılıyor, asalsa kalanın da asal olup

olmadığı aynı fonksiyonla kontrol ediliyordu. Örneğin 20 için 3'e bakılır. 3 asaldır, 20'den 3 çıkarılır, 17 de asal mı diye denetlenir. İkisi de doğruysa çift yazdırılır, bayrak 0 yapılır ve `while`'dan çıkılır. Bir kere yazdırılır, başka çift aranmaz.

> **Ders sonrası not:** Öğrenci yarıya kadar gitmesinin sebebini "yarısından sonra bir bölen çıkmıyor" diye anlattı. Burada aranan bölen değil toplam çifti olduğu için doğru gerekçe büyük olasılıkla şudur: Yarıdan sonraki adaylar aynı çiftleri ters sırayla tekrar eder.

Hoca, bayrağın `flag == 1` şartı yerine asallık koşullarının tersinin doğrudan döngü şartına yazılabileceğini söyledi. Bunun için "ve" ile bağlı şartların tersini almak, yani bağlacı "veya" yapmak gerekir: `!asal(x) || !asal(i - x)` ve üçüncü şart olarak `x > i - x`. Hoca öğrenciden bunu denemesini istedi.

Asallık fonksiyonunda öğrenci, karekökle ve virgüllü sayılarla uğraşmamak için `i * i <= sayi` koşulunu kullanıyordu. Bölünüyorsa fonksiyon yanlış (0), bölen yoksa doğru (1) döndürüyordu. Program çalıştırılırken ekranda bir şey görünmeyince hoca bir ilkeyi hatırlattı.

> **Hocanın vurgusu:** Bir veri girişi yapılacaksa önce `printf` ile ne girileceğini söyleyin, sonra `scanf` kullanın.

Bu kodun çıktısı da olumlu oldu. Hoca, o ana kadar hiçbir öğrencinin kodunda olumsuz sonuç çıkmadığını belirtti. Öğrenci, kodunun dizi kullananlardan hızlı olmadığını da ekledi.

8. Algoritmaları Karşılaştırmak

8.1 Zaman Ölçümü

Dört öğrencinin dört farklı algoritması olduğuna göre bunlara A1, A2, A3, A4 diyelim. Örneğin $N = 100.000$ için hangisinin daha iyi olduğunu nasıl kıyaslarız? Öğrencilerin ilk cevabı çalışma zamanına bakmaktı. Bir zamanlayıcı (timer) kullanılır: Kodun çalışmaya başladığı zaman ile bitirdiği zaman arasındaki farka (time difference) bakılır. Bu sırada `printf`'ler kapatılır, aradaki döngüler yalnız başına çalışır.

Peki her öğrenci bunu kendi bilgisayarında mı ölçmeli? Hayır. Herkesin işlemcisi, RAM'i, derleyicisi, işletim sistemi ve daha pek çok şeyi farklıdır. Koşullar aynı tutulmalıdır: Bütün kodlar tek bir bilgisayara yüklenir ve $N = 100.000$ için A1, A2, A3, A4 aynı bilgisayarda çalıştırılır.

8.2 Aynı Bilgisayarda Bile Yetmez

Hoca bunun da bir yerden sonra çok şey ifade etmediğini söyledi. Bu proses çalışırken işletim sisteminde başka birçok proses de çalışır. İşletim sistemi bir prosesin önceliğini artırıp diğerininkini azaltmış olabilir. Böylece A1'in A2'den daha geç bitmesi beklenirken daha erken bitebilir. Bu yüzden zaman ölçümü, yani deneysel analiz (empirical analysis), kaba bir analiz sayılır. Hoca, yalnızca bizim programımızın çalıştığı işletim sistemsiz bir sistemde 10.000 veya 100.000 eleman için süre ölçmenin belki anlamlı olabileceğini, ama günümüzde bütün sistemler işletim sistemiyle yönetildiği için bunun pek kullanılmadığını ekledi.

8.3 Temel İşlem Sayısı

Daha objektif ölçüt, **çalışma zamanı analizidir (running time analysis)**. $T(N)$ ile gösterilen bu değere adım sayısı (number of steps) da denir. Ölçüm zamanlayıcıyla değil, **temel işlem sayısı (number of basic operations)** ile yapılır.

Hoca önceki derslerden e^x örneğini hatırlattı:

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + ...$$

Orada iç içe iki döngü oluşuyordu. Her seferinde iç döngüyü yeniden kurmak yerine elde edilen değerler korunarak tek döngüye inildi ve adım sayısı azaldı. Temel işlem asıl yükü çeken şeydir, zamanı da o belirler. Bu yüzden algoritmaları şöyle bir tabloyla kıyaslamak zaman ölçümünden daha anlamlıdır. Hocanın $N = 100.000$ için temsilî olarak verdiği değerler şunlardır:

Algoritma	Temel işlem sayısı
A1	100.000
A2	50.000
A3	200.000
A4	10.000

Bu temsilî değerlere göre en az işlem A4'te, en çok işlem A3'tedir (ara adım eklendi).

Soru: İşletim sisteminin gösterdiği "Energy Impact" (enerji etkisi) değeri önemli mi? Görev yöneticisine benzer ekranda CPU ve belleğin yanında, altında disk ve ağ kullanımıyla birlikte görünüyor. **Cevap:** Bunlar da bir etkidir, ama enerjiyi asıl harcayan işlemcidir. Etki küçüktür ve anlamlı olması için istatistiksel test gerekir: Bir programın enerji etkisi ile diğerininki arasında anlamlı fark var mı, onu sınamak gerekir. Anlamlı olan, bir algoritmada temel işlemin ne kadar çok yapıldığıdır. Algoritma analizi çalışma zamanı ve bellek verimliliği üzerine kuruludur.

Hoca güç kaynağı gibi dış etkenlerin de tamamen etkisiz olmadığını söyledi. Güç kaynağı yetersiz kalırsa işlemcide performans düşer, proses yavaşlar. Ama bu dolaylı bir etkidir ve diğer prosesler için de aynen geçerlidir. Hoca bu konuyu Algoritma Analizi dersinde daha ayrıntılı anlattığını da belirtti.

9. Mesleki Derslere Yaklaşım ve Derleyici Farkları

Aradan sonra hoca, derse katılımla öğrenmenin önemini anlattı. Etkileşimin en çok katılımla oluştuğunu söyledi.

> **Hocanın vurgusu:** Mesleki derslerde, özellikle algoritma ile ilgili temel (core) derslerde, "ileride bunu birine öğretirim veya bununla bir proje yaparım" düşüncesiyle öğrenmeye çalışın.

Soru: macOS'ta C kodu çalıştırırken `int main()` içine `void` yazmayınca sürekli hata alıyorum. **Cevap:** Derleyici orada parantez içine bir şey yazılmasını istiyor. Hocanın derleyicisi buna gerek duymuyor. Mac'teki derleyici belki C89 veya farklı bir derleyicidir. Derleyicinin standart ayarı değiştirilirse sorun gider. Yine de kodların derste yazıldığı biçimde yazılması istenir, çünkü dikkat edilecek 2-3 temel kural vardır.

> **Ders sonrası not:** Hoca bu durumu olası bir C89 ayarıyla açıkladı. Ders sonrasında `int main()` biçimi `gcc -Wall -std=c89 -pedantic` ile hatasız derlendi. Bu yüzden Mac'teki durumun sebebi büyük olasılıkla C89 değil, derleyicinin katı uyarı veya hata ayarlarıdır.

10. En Büyük Not Problemi

Hoca veri yapılarına geçişi hazırlamak için basit bir problem yazdı: N öğrencinin notu girilecek, en büyüğü bulunacak. Burada yalnızca algoritmanın üç temel yapısı bilindiği kabul edilir: sıralı yapı, şartlı yapı ve tekrarlı yapı. Diziler henüz kullanılmayacaktır.

Soru: `grade` neden dizi olarak tanımlanmadı? **Cevap:** Diziyi bilmiyoruz, öğrenmedik. N tane sayı gireceğiz ve en büyüğünü bulacağız.

10.1 Değişkenler

Değişkenler `grade`, `N`, `max` ve döngü değişkeni `i`'dir. Hoca not için `not` adını kullanmadı, gerekçe olarak bunun ayrılmış kelime (reserved keyword) olduğunu ve sorun çıkardığını söyledi.

> **Ders sonrası not:** Hoca `not` adının ayrılmış kelime olduğunu söyledi. Ders sonrasında `int not=5;` C'de `gcc -Wall -std=c89 -pedantic` ile hatasız derlendi, `g++` ile ise hata verdi. Yani `not` C'de büyük olasılıkla ayrılmış kelime değildir, C++'ta ise bir işleç adı olarak ayrılmıştır. `grade` adını kullanmak yine de sorunsuzdur.

10.2 `for` İçinde Değişken Tanımlamak

Hoca kodu yazarken sık yapılan bir hatayı göstermek için `i`'yi bilerek `for` içinde tanımladı:

```
for (int i = 1; i <= N; i++)
```

Derleyici şu hatayı verdi: `'for' loop initial declarations are only allowed in C99 mode`. Kod bu hâliyle derlenmez, dolayısıyla çalıştırılamaz. Hoca bunun C99 standardında izin verilen bir şey olduğunu, ama bu derste kullanılan standartta olmadığını söyledi.

> **Hocanın vurgusu:** Döngü değişkenini yukarıda tanımlayın, `for`'un içinde tanımlamayın. Başka dillerde yapabilirsiniz ama bu derste yapmayın. Önemli bir şey.

10.3 En Büyüğü Bulmak

Döngü 1'den N'e kadar döner, çünkü dizi bilinmiyor ve "birinci öğrenci, ikinci öğrenci" diye gidiliyor. Bir sayı dizisinde en büyüğü bulmayı bir öğrenciye soran hoca şu cevabı aldı: İlk değeri en büyük kabul edip diğerleriyle karşılaştırmak. Burada başlangıç değeri olarak 0 verilir, çünkü notlar sıfırdan küçük olmaz. Girilen değer `max`'tan büyükse `max`'a atanır:

```
if (grade > max)
    max = grade;
```

Hoca sınıfa bu kodun problemi çözüp çözmediğini sordu. Bir öğrenci "çalıştıralım, olmazsa düzeltiriz" dedi. Hoca bunu kabul etmedi.

> **Hocanın vurgusu:** Kodu okuyabilmek gerekir.

Kodu birlikte okudular: `i` 1'den başlar, döngü N kez döner ve $N + 1$ kez kontrol yapılır. Bir not girilir, `max`'tan büyükse saklanır. Bir öğrenci en yüksek notu alanın adını yazdırmayı önerdi. Hoca adı, soyadı bir kenara bırakıp probleme odaklanmayı, soruya eklemeler yapmamayı istedi.

10.4 Negatif Notu Önlemek

Öğrencilerden gelen itiraz, negatif not girilebileceğiydi. Hoca bunu kabul edip not okumayı `do-while` içine aldı. En az bir kez not okunacağı için `do` uygundur. Bu hâli gcc ile derlenmiş tam kod şöyledir:

```
#include <stdio.h>
int main(){
    int grade,N,max=0,i;
    printf("Ogrenci sayisi=");
    scanf("%d",&N);
    for(i=1;i<=N;i++){
        do{
            printf("%d. ogrencinin notu",i);
            scanf("%d",&grade);
        }while(grade<0);
        if(grade>max)
            max=grade;
    }
    printf("Maksimum not=%d",max);
    return 0;
}
```

Hoca bu kodu adım adım okuttu. `i = 1` için 70 girildi. 70 sıfırdan küçük değildir, döngüden çıkılır. 70 büyüktür `max` (0), o hâlde `max` 70 olur. `i` 2 olur ve yeni not istenir. -30 girildi. -30 sıfırdan küçüktür, döngü başa döner ve aynı öğrencinin notu yeniden istenir. Böylece negatif değer kabul edilmez.

Soru: Eksi not olmadığına göre kullanıcının yanlışlıkla eksiye bastığı varsayılıp mutlak değer alınamaz mı? **Cevap:** O da olur. Ama belki kullanıcı 30 yerine yanlışlıkla -20 girmiştir. Bu işin sonu yoktur. Hocanın önerisi, aynı öğrenci için notu yeniden girmektir.

10.5 En Büyük Notun Sahibi

Hoca notun kaçıncı öğrenciye ait olduğunu da saklamak istedi. Bunun için `max_i` değişkeni eklendi ve `if` içine `max_i = i;` yazıldı.

```
if (grade > max) {
    max = grade;
    max_i = i;
}
```

Derste 5 öğrenci için 2, 3, 6, 1, 4 notları girildi (notlar 10 üzerinden). Adımlar şöyledir (ara adım eklendi):

i	grade	grade > max?	max	max_i
1	2	Evet (2 > 0)	2	1
2	3	Evet (3 > 2)	3	2
3	6	Evet (6 > 3)	6	3
4	1	Hayır	6	3
5	4	Hayır	6	3

Sonuç: En büyük not 6'dır ve 3. öğrenciye aittir. Program da aynı sonucu verdi.

Öğrenciler `return 0` yazılmasını da hatırlattı. Hoca bunun çok önemli olmadığını, program tamamlanınca yalnızca bir mesaj göndermeye yaradığını söyledi. Negatif girişler de denendi: Dokuz öğrencili denemede -3 birkaç kez reddedildi, 0 ise geçerli bir not olarak kabul edildi.

10.6 Eşit Notlar

Aynı denemede 7. ve 8. öğrencilerin ikisi de 5 aldı. Program "en büyük not 5, bu not 7. öğrencinin" dedi. 8. öğrenciyi almadı, çünkü koşul `grade > max` idi ve 5, 5'ten büyük değildir. `>=` yazılsaydı 8. öğrencinin kaydı alınırdı. Ders sonrasında derlenen kodda iki durum da böyle sonuç verdi.

10.7 Döngü Sonunda i'nin Değeri

Soru: Döngüden sonra `i` yazdırılsa değeri N'e bağlı olarak ne olur? **Cevap:** $N+1$. `i = N` için döngü şartı sağlanır, içerideki işlemler yapılır ve `i` bir artırılır. `i = N + 1` olunca şart sağlanmaz ve döngüden çıkılır.

10.8 Bir Öğrencinin İtirazı

Bir öğrenci, iki kişinin notu aynı olursa ne olacağını sordu. Hoca bunu önemli bir sorun olarak aldı. Birbirinden farkı olmayan iki öğrenciden yalnızca birini yazmak çok adil değildir. İlk görülen (first occurrence) veya son görülen (last occurrence) kaydı yazmak yetmez, ikisini de yazmak gerekir. Hoca bu soruyu "aklımızın bir köşesinde dursun" diyerek bir kenara bıraktı.

11. En Büyük İkinci Not

Hoca soruyu genişletti: En büyük ikinci notu da bulalım. Bir değişken daha gerekir: `max2` ve yerini tutan `max2_i`. `max2` de 0 ile başlar.

Girilen bir not için üç durum vardır: en büyükten büyük olabilir, en büyük ikinciden büyük olabilir veya ikisinden de küçük olabilir. Bir öğrenci önemli noktayı önceden söyledi: Yeni `max` gelince eski `max`'ı `max2`'ye atamak yeterli olmaz mı? Hocanın tahtada çizdiği akış şudur:

1. `grade > max` doğru mu?
2. Doğruysa yeni bir en büyük bulunmuştur. Önce eski `max` yedeklenir: `max2 = max`. Sonra `max = grade` yapılır.
3. Yanlışsa `grade > max2` doğru mu diye bakılır.
4. Doğruysa `max`'a dokunmadan `max2 = grade` yapılır. Hocanın deyimiyle "ikinci sıradaki vatandaş yok oldu".

5. O da yanlışsa hiçbir şey yapılmaz.

Yerleri de aynı sırayla kaydırmak gerekir. Hoca önce `max2_i`'ye yanlışlıkla `i`'yi yazar gibi oldu, bir öğrencinin uyarısıyla düzeltti: "Şu i'ler kafamızı karıştırdı." Doğru sıra önce `max2_i = max_i`, sonra `max_i = i` biçimindedir.

11.1 Başlangıç Değeri Tartışması

Bir öğrenci, ilk `if`'e girildiğinde `max_i`'nin henüz bir değeri olmadığını, bu yüzden `max2_i = max_i` atamasının sorunlu olduğunu söyledi. Hoca önce değerin nasılsa atanacağını düşündü, sonra itirazı kabul etti.

Soru: Varsayılan değeri 0 değil mi? **Cevap:** Varsayılan değeri bir kenara bırakın. Değişkende o an ne varsa o kalır.

Çözüm olarak iki indis de başlangıçta sıfırlandı: `max_i=max2_i=0;`. Bu satır sağdan sola işler. Önce 0 değeri `max2_i`'ye atanır, atama bir değer döndürür, dönen bu değer de `max_i`'ye atanır.

11.2 Son Kod

Derste yazılan kodun son hâli şöyledir:

```c
#include <stdio.h>
int main(){
    int grade,N,max=0,max2=0,max_i,max2_i,i;
    printf("Ogrenci sayisi=");
    scanf("%d",&N);
    max_i=max2_i=0;
    for(i=1;i<=N;i++){
        do{
            printf("%d. ogrencinin notu",i);
            scanf("%d",&grade);
        }while(grade<0);
        if(grade>max){
            max2=max;
            max2_i=max_i;
            max=grade;
            max_i=i;
        }else if(grade>max2){
            max2=grade;
            max2_i=i;
        }
    }
    printf("Max Not=%d, bu not %d. ogrencinindir",max,max_i);
    printf("Max_2 Not=%d, bu not %d. ogrencinindir",max2,max2_i);
    printf("%d",i);
    return 0;
}
```

Derste 5 öğrenci için 3, 4, 2, 1, 5 notları girildi. Adımlar şöyledir (ara adım eklendi):

i	grade	Hangi kol	max	max_i	max2	max2_i
1	3	grade > max	3	1	0	0
2	4	grade > max	4	2	3	1
3	2	Hiçbiri (2 < 3)	4	2	3	1
4	1	Hiçbiri	4	2	3	1
5	5	grade > max	5	5	4	2

Sonuç: En büyük not 5 (5. öğrenci), en büyük ikinci not 4 (2. öğrenci). Döngü sonunda `i` değeri 6'dır. Kod `gcc -Wall` ile derlendi ve aynı girdiyle aynı sonucu verdi.

12. Algoritmanın Tıkandığı Yer

Hoca soruyu bir kez daha genişletti: En yüksek 6. notu istiyorum. Bir öğrenci aynı işlemleri 6'ya kadar tekrar etmeyi önerdi. Hoca burada algoritmanın tıkandığını söyledi, daha doğrusu çözümün "hamallıktan başka bir şey" olmadığını. `max1`, `max2`, …, `max6` değişkenleri ve bunların indisleri gerekir. 6 belki yazılır, ama 60. not istense 60 değişken ve 60 indis olur. Altta da "60'tan büyükse, 59'dan küçükse…" diye uzayan bir karşılaştırma zinciri gelir.

Aslında bu problem çözülür. 100 öğrenci için 100 değişken tanımlanır, kod uzun olur ama çalışır. Hoca "programlama bu değildir" dedi, bunu elle hesap makinesinde veya Excel'de de yapabiliriz.

İkinci sorun, eşit notlar sorusudur. En yüksek notu alan birden fazla öğrenci varsa hepsi yazılmalıdır. Hoca bunun bu yaklaşımla imkânsız olduğunu örnekle anlattı: İlk öğrenciye 100 girildi, son öğrenciye de 100 girildi. Algoritma buna göre tasarlandıysa ilk öğrenci unutulmuştur. Notlar sürekli tek bir değişkenin üzerine yazılır.

Algoritmik yapıların üçü de (sıralı, şartlı, tekrarlı) burada çözüm sunmaz. Notları bellekte saklamak gerekir. Bunu da programlama dilinin ek bir araç olarak sunması gerekir.

13. Veri Soyutlama ve Dizi

Burada hoca **veri soyutlama (data abstraction)** konusunu açtı. Slaytların genel hâlini daha sonra anlatacağını söyledi. Günlük hayattaki problemleri bilgisayarla çözeriz. Verilerin RAM'de veya başka bir hafıza biriminde saklanmasını istiyorsak programlama dili bunu kolaylaştırmak zorundadır. Bir dilde dizi tanımlanamıyorsa da verileri saklamak için başka bir yapı olmak zorundadır.

Hoca "Diziler olmasaydı C'de nasıl yapabilirdik?" diye sordu. Bir öğrenci birbirine bağlanan yapıları önerdi. Hocanın cevabı şuydu: Bağlı liste de bir yapıdır, ama o da bir structure türü ve pointer uygulamasıdır. Her dilde bağlı liste diye hazır bir şey yoktur. Bağlı liste, bellek hücrelerini birbirine bağlamayı gösteren bir yapıdır.

Hoca **veri yapısını (data structure)**, verinin bellek üzerinde, özellikle RAM üzerinde organize edilmesini sağlayan mantıksal yapılar ve algoritmalar olarak tanımladı. Bunun sabit disk sürümü de vardır. İkinci sınıfta bunun ayrı dersi anlatılacaktır.

> **Ders sonrası not:** Hoca "data abstraction" ile "data structure" terimlerini aynı anlatım içinde kullandı. Doğrusu büyük olasılıkla bunların ayrı kavramlar olmasıdır: Veri yapısı verinin organizasyonudur, veri soyutlama ise bu organizasyonun ayrıntısını gizleyip yalnız işlemlerini göstermektir.

13.1 Dizi

En basit veri yapısı dizidir. Belleğin fiziksel adresleri vardır. `int a[10];` yazıldığında bellekte bir noktadan başlayarak ardışık hücreler üretilir. `a`, 0. elemanın adresini verir. `a[0]`'ın kendine ait bir adresi, `a[1]`'in de kendine ait bir adresi vardır.

> **Ders sonrası not:** Hoca 0. elemanın adresinin `a`'nın içine kopyalandığını söyledi. Doğrusu büyük olasılıkla şudur: `a` adresi saklayan ayrı bir değişken değildir, ifadede ilk elemanın adresine dönüşür. Ders sonrasında `int a[10];` için `sizeof(a)` 40 bayt çıktı ve `a` ile `&a` aynı adresi gösterdi.

Dizi tek boyutludur (one dimensional), çünkü RAM'in kendisi de tek boyutludur: Ardışık sıralanmış mantıksal bir yapıdır. Bu yapıya iki türlü erişilir:

- **Sıralı (sequential) erişim:** Döngüyle `a[i]` kullanarak 0'dan n'e kadar gitmek.
- **Rastgele (random) erişim:** `a[5]`, `a[8]` gibi doğrudan bir elemana gitmek.

İndis yeni bir değişken gibi görünse de aslında arka planda bir adres hesabı vardır. Hoca bunun programlamanın sağladığı bir kolaylık olduğunu, algoritmada normalde böyle bir şeyin bulunmadığını söyledi. Assembly dilinde de yardımcı elemanlarla bellek üzerinde gezinilir.

14. Otomata Modelleri ve RAM Modeli

Hoca, araştırmaları için otomata teorisini önerdi ve ikinci sınıfta bunun dersi olduğunu söyledi. Otomata teorisi küçük bellekler kullanır, ama bu belleklere istediğiniz gibi erişemezsiniz. Örneğin bir modelde bellek hücrelerinin üzerinde yalnızca sağa doğru gidebilirsiniz, sola gelemezsiniz. Veri kaydetme imkânı vardır, ama sağa doğru gidilir ve boş karaktere ulaşınca işlem biter. Program bu mantık üzerine kurulur.

Hoca bir başka basit model olarak turnikeleri örnek gösterdi: Para atılır, sayılır. Geçersiz bir şey girilirse geçersiz olur. Para tanınmazsa makine farklı bir duruma düşer.

Bizim programlamada kullandığımız model ise **RAM modelidir (Random Access Memory model)**. Dizilerin, iki boyutlu matrislerin ve bunların farklı biçimlerinin üzerinde işlem yaparız ve istenen hücreye doğrudan erişiriz.

15. Dizi Bize Ne Kazandırır?

Dizi sayesinde girilen not tek bir değişkenin üzerine yazılmaz, kalıcı olarak saklanır. Hoca örnek olarak dizide 10, 20, 30, 40 notlarını düşündü. Eşit notlar sorunu artık iki geçişle çözülür:

1. Önce en yüksek not bulunur. Burada 40'tır.
2. Dizinin başından itibaren bir kez daha gezilir.
3. 40 nerelerde geçiyorsa o yerler saklanır veya doğrudan ekrana yazdırılır.

Hocanın sorusuna gelince, en yüksek 40. kişiyi bulmak için de dizi üzerinde algoritma geliştirilebilir. Öğrenciler sıralamayı önerdi. Hoca 1000 eleman üzerinden üç yolu karşılaştırdı:

- **Bütün diziyi sıralamak:** Bir çözümdür, ama 40. elemanı bulmak için 1000 elemanı sıralamak gerekir.
- **Yalnızca ilk 40 elemanı bulmak:** En küçük 40 tanesi bulunur ve arama bırakılır. Bu, 1000 elemanı sıralamaktan daha iyidir. Süresi elemanlara göre değişir.
- **Daha hızlı sıralama algoritmaları:** Hoca "quick search" adını kullandı ve bunları özyinelemeli (recursive) algoritmalar kısmında göreceklerini söyledi.

  Ders sonrası not: Hoca hızlı sıralama algoritması olarak "quick search" dedi. Sıralama bağlamında doğrusu büyük olasılıkla "quick sort"tur.

Hoca dersin sonunda bu konuya bir kez daha döndü. Bazen sıralama gerekli olmayabilir, hızlı arama algoritmaları kullanılabilir. Burada kastettiği ikili arama değildir, çünkü ikili arama sıralanmış bir veri üzerinde arama yapar.

16. Matris, Graf ve Ağaç

16.1 Matris

Veri iki boyutluysa matris kullanılır. Örneğin birinci boyut öğrenciler, ikinci boyut sütunlardaki vize, final, ödev ve ortalama alanlarıdır. Bu dört alanın hepsi `double` olabilir. Ortalama sütunundan en yüksek ortalama bulunur, oradan o öğrencinin diğer bilgilerine ulaşılır.

Ama bu veri belleğe yüklendiğinde yine tek boyutludur. Hoca bunu sayısal olarak gösterdi. 16 GB RAM olsun:

$$16 \text{ GB} = 2^4 \cdot 2^{30} = 2^{34} \text{ bayt}$$

Hücreleri 8 bitlik, yani 1 bayt kabul edersek adresler 0'dan $2^{34} - 1$'e kadar tek tek uzanır. Matrise 1, 2, 3, 4, 5, 6, 7, 8 değerleri girilse bunlar bellekte aynı sırayla ardışık durur. İki boyutlu bir görünüm yoktur. `M[i][j]` yazarak ulaşmak, başlangıç adresinden itibaren yapılan bir adres hesabıdır.

Hocanın örneği 5 değeridir: `M[1][0]`. Önce birinci satıra, sonra o satırın 0. sütununa gidilir. Tahtadaki düzen 2 satır ve 4 sütun ise hesap şöyledir (ara adım eklendi): $1 \cdot 4 + 0 = 4$, yani baştan 4. indisteki hücre, değeri 5. Ayrıntısı pointer'lar konusunda anlatılacaktır.

16.2 Graf

Hoca grafı şehir örneğiyle anlattı: İstanbul, Kocaeli, Bolu, Ankara, Eskişehir, Bursa, Yalova, Kütahya gibi şehirler ve aralarındaki yollar. Her yolun bir maliyeti vardır. İstanbul'dan Ankara'ya birçok yoldan gidilebilir. En ucuz, en kısa veya en hızlı yol istenebilir. Bunu bir algoritma bulur, ama önce şehirleri ve aralarındaki bağlantıları bellekte saklamak gerekir.

> **Ders sonrası not:** Hoca "Ankara bir graftır" dedi. Doğrusu büyük olasılıkla Ankara'nın grafın bir düğümü olmasıdır, graf şehirlerin ve yolların tamamıdır.

Hoca çizimde 9 şehir saydı ve 9×9'luk bir matrisle hangi şehrin hangi şehre ne kadar uzak olduğunu sakladı. Yollar iki yönde aynı maliyetliyse, yani İstanbul'dan Kocaeli'ye maliyet Kocaeli'den İstanbul'a maliyetle aynıysa, matris simetriktir. Bu veri üzerinde en kısa yol algoritması çalıştırılarak nereden nereye ne kadar sürede gidilebileceği hesaplanır. Hoca "her şey dizilerden ve matrislerden türüyor" dedi.

16.3 Ağaç

Ağaç, hiyerarşik bir yapıdır. Bir okulda rektör, onun altında dekanlar, onların altında bölümler ve her fakültedeki öğrenci sayıları gibi. Bu yapı da tek boyutlu bir dizide saklanabilir: 0. eleman kök olur, sonraki üç eleman onun çocuklarıdır. Ağaç düzenliyse, yani her düğümün üçer veya ikişer çocuğu varsa dizide saklamak kolaydır. Düzenli değilse dizilerin biraz farklılaşmış hâli olan bağlı liste mantığı kullanılır.

17. Otomata Türleri ve Veri Yapılarının Önemi

Hoca, otomata teorisine dönerek RAM'de elimiz kolumuz serbest yaptığımız işlemleri bu modellerde yapamayacağımızı söyledi. En basit bilinen otomata makineleri şunlardır:

- Deterministik sonlu otomat (Deterministic Finite Automata, DFA)
- Deterministik olmayan sonlu otomat (Non-Deterministic Finite Automata, NFA)
- Yığıtlı otomat (Pushdown Automata)
- Turing makinesi

Bunlar da bilgisayarımız gibi birer makinedir. Hocanın anlatımıyla DFA ve NFA'ya 0 ve 1'lerden oluşan bir giriş verilir, girişte tek yönde ilerlenir ve sonuçta doğru mu yanlış mı olduğu söylenir. Pushdown Automata'da biraz ileri geri hareket etme kabiliyeti vardır, Turing makinesi daha kabiliyetlidir. Hoca programlama dillerinin ve birçok algoritmanın bu modellerden geliştirildiğini söyledi ve konuyla ilgili videolara bakılmasını önerdi.

> **Ders sonrası not:** Hoca Pushdown Automata'nın ileri geri hareket edebildiğini söyledi. Doğrusu büyük olasılıkla şudur: Bu makinenin farkı ek bir yığın (stack) belleği olmasıdır, girişte iki yönlü hareket Turing makinesinin özelliğidir. Ayrıca hoca girişin yönü için bir yerde "sağa doğru", bir yerde "sağdan sola" dedi; asıl nokta girişin tek yönde okunmasıdır.

Hoca konuya neden girdiğini şöyle bağladı: Algoritmada öğrendiğimiz `for`, `if`, `while` gibi üç temel yapı problemleri çözmeye yetmez. Eşit notlar problemi bu yapılarla çözülemez, k. en büyük problemi ise çözümü anlamsız derecede uzatır.

> **Hocanın vurgusu:** Bu derste diziler, matrisler ve bağlı listeler görülecektir. Ağaçlar ve graflar dahil Veri Yapıları dersinde anlatılanların hepsi dizilerden ve matrislerden oluşur. Temel bu olduğu için bunları çok iyi öğrenmek gerekir.

18. Adresler ve Çok Boyutlu RAM Soruları

Soru: Değişkenlerin adreslerine rahatça erişebiliyoruz. Bu adresler de bir yerde kayıtlı olmalı. Bilgisayar bunu nasıl biliyor? **Cevap:** Değişkenlerin adresleri bellekte tutulmaz, değerleri tutulur. `a` yazmak aslında bir adres üzerinden erişmektir. Adres saklanmak istenirse o da bir değişkendir ve bir veri türüdür. Örneğin `a[0]` içinde 10 saklanırken, onun adresi 100 ise bu 100 de başka bir yerde saklanabilir. Adres yolu (address bus) ise çalıştırılacak programın ilgili satırının adresini getirip götürür. Değişken oluşturulunca bellekte bir yer ayrılır, adresi pointer'lar ile elde edilebilir. Konu pointer'larda netleşecektir.

> **Ders sonrası not:** Hoca soruya önce “RAM’de saklanıyor” dedi, sonra adres yolundan söz etti. Doğrusu büyük olasılıkla şudur: Değişkenin yerini derleyici ve bağlayıcı belirler, adres veya göreli konum makine kodunun içinde yer alır. Yani ayrı bir adres tablosu tutulmaz. Adres yolu ise o adresi işlemciden belleğe taşıyan donanım yoludur.

Soru: RAM tek boyutlu dediniz. İki veya üç boyutlu RAM modelleri var mı? **Cevap:** Hocanın bildiği kadarıyla fiziksel olarak yok. RAM’ler kaç GB olursa olsun 0. adresten son adrese kadar tek boyutlu adreslenir. Fiziksel olarak hücrelerin nasıl saklandığını bilmediğini, donanımla uğraşanların daha iyi bilebileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Fiziksel DRAM yongalarında hücreler büyük olasılıkla satır ve sütunlardan oluşan iki boyutlu dizilimler hâlinde bulunur. Bellek denetleyicisi tek boyutlu adresi bu satır ve sütun bilgisine çevirir. Programcının gördüğü adres uzayı yine tek boyutludur.

Hoca işlemci tarafında ise farklı bir durum olduğunu ekledi: Bir diziyi başka bir diziyle, bir matrisi başka bir matrisle, hatta üç boyutlu bir diziyi başka bir üç boyutlu diziyle `for` kullanmadan tek seferde işlemek mümkündür. Bu işlem elemanı tarafındadır, bellek tarafında iki veya üç boyutlu bir işlem olup olmadığını bilmediğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Hocanın anlattığı tek seferde işleme büyük olasılıkla vektör işlemeye veya SIMD (Single Instruction, Multiple Data) buyruklarına karşılık gelir. Bu buyruklarda tek bir komut birden çok veri üzerinde aynı işlemi yapar.

Dersin Toparlaması

Dersin ilk yarısında dört farklı Goldbach kodu üzerinden asal deposu, karekök sınırı, gereksiz bayrağın kaldırılması, ikili arama ve bellek-zaman ödünleşimi görüldü. Büyük dizilerde alınan hata, prosesin bellek bölgelerine ve statik ile dinamik bellek ayrımına kapı açtı. Dört algoritmanın karşılaştırılması ise zaman ölçümünün kaba bir analiz olduğunu, asıl ölçütün temel işlem sayısı olduğunu gösterdi.

İkinci yarıda en büyük not probleminden en büyük ikinci nota geçildi, sonra algoritmanın üç temel yapısının 6. en büyük notta ve eşit notlarda neden yetmediği görüldü. Hoca bu örneği, algoritmanın her zaman tek başına çözüm olmadığını ve veri yapılarının önemini göstermek için seçtiğini söyledi. Sıralama ve hızlı arama algoritmaları, özyinelemeli algoritmalar, pointer’lar, adres hesabı ve bağlı listeler ileride bu kapıdan girilerek görülecektir. Hoca gelecek derste de bu konudan devam edeceğini belirtti.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.